**Notes for myself when brainstorming and planning out the implementation:**

- n devices
    - Each recording n audio inputs
- Latency: real-time
    - Since its real time, we cant be constantly calling api methods, so websocket connections are better for constant streaming
- Tool/service inputs:
    - n > 1 live audio streams
- Output: Single audio stream of the combined inputs
- Why do we need multiple recording files, if its all the same real time stream?
    - Is it cause some voices are more clear on different devices?
- What exactly is audio mixing?
    - Basically when having multiple audio sources, we want to “mix” them together or add together the sound waves to get a unified complete audio, that incorporates all the source audios.
    - Since everything is real time both streams should be exactly the same or close enough being the same, where adding them up should just amplify any audio sources.
- “It must be designed as a convenient, out-of-the-box solution that is easy for other developers to integrate and use.”
    - Maybe some sort of api service? Or public functions?
- Dont need a database but we do need a local memory or state
- Frontend:
    - “Multiple devices (clients) to dynamically join, record, and leave.”
        - Maybe a room where people can input a room code to join
            - Joining a room should start a websocket
        - A “record” button for each user to start recording audio
            - Should also be an api method
        - Real-time monitoring and playback of the final mixed audio stream.
            - What do they mean by real time monitoring? Is it transcribing what the users have said? Should there be an indicator showing that the audio is being recorded?
            - User can end recording and get a finalized audio file
- To keep things modular or more organized, separate the directories into one for the core module, one for the ui/frontend, and one for the backend server
- Would probably need to normalize or keep all audio files in the same format:
    - Sample rate: Highest frequency for speech is 8 kHz, so because of nyquist theorem (sampling frequency ≥ 2 * max frequency)
    - Bit depth: int16 (what websocket and wav uses natively)
    - Channels: mono
    - Frame size: 20 ms (found is standard in WebRTC)
- How can I simulate and test multiple voices?
    - Can create audio files with elevenlabs and test that without using a mic

    - Can create environments similar to real world experiences, for example can create fake networking issues or delays to simulate the real world.

Edge cases or faulty scenarios:

- Since we are adding sound waves, we need to clamp the loudness of the voices, or else they will overflow
- **Zero participants.** The loop still ticks. `mix_frames([])` must return silence, not crash or return an empty array of the wrong length.
- In the core mix module, a frame can arrive for a device after they have left.
- Devices could have the same name (not much of a problem since it wont break anything, just confusing)
- Devices can disconnect accidentally or uncleanly
- Last participant leaves while recording
- Empty or whitespace-only device names
- Websocket closes mid send
- If devices are outputting audio from what they are recording and translating, it can create feedback loop

V0 architecture: Start off with non realtime audio inputs

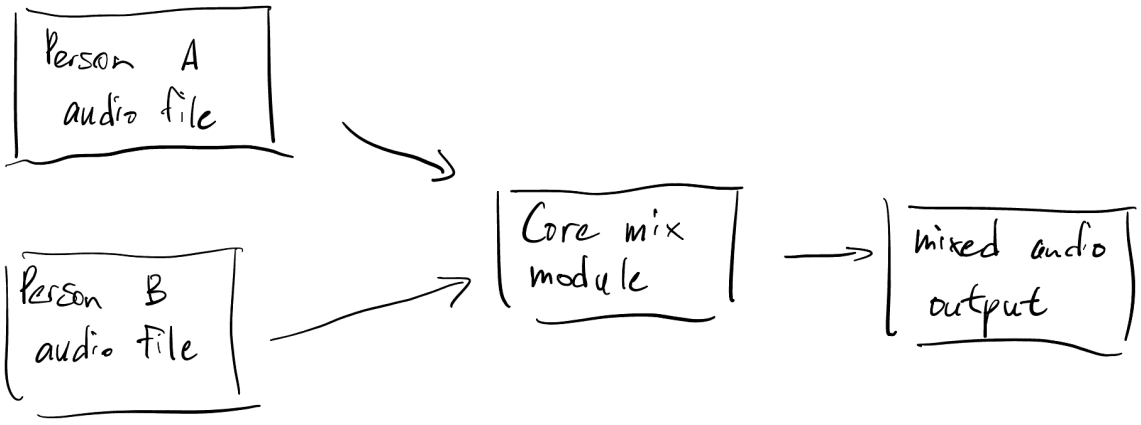
- Take two audio files, and be able to combine them into a unified stream to simulate real time processing
- Reads two wav files, and outputs one wav

V2 architecture: Now be able to take two real time audio streams and combine them into one

- How can I test this, if I am the only one? Maybe simulate by having two elevenlabs audio files having a conversation, and process them in real time or pretend to atleast.
- How do we simulate the audio files in real time?
    - Concatenate the audio streams in chunks? Sampling time?
- Feed the same files through a fake clock at 1x speed via the streaming api

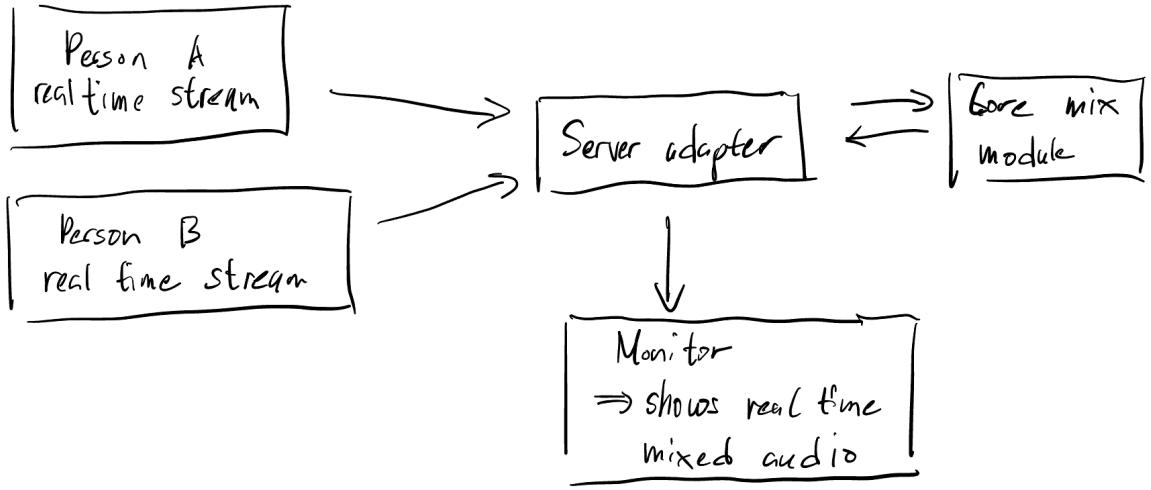
**Final Iteration designs:**

- **V0:** Read two WAV files, split both into aligned 20ms frames, sum frame by frame with a clamp, and write one mixed WAV you can listen to.




- **V1:** Get audio flowing live from two browser tabs through a WebSocket to a third monitor tab, mixing on arrival with one slot per device.
    - Implementation steps:
        - 1. FastAPI app, plus a WebSocket endpoint that says back whatever it receives. Open the page, connect from the console, send "hello", see it come back.
        - 2. Test on synthetic audio
        - 3. Second WebSocket route. Server forwards every chunk it receives straight to any connected monitor, no mixing yet.
        - 4. Server keeps a dictionary of each device and its latest audio chunk. On each arrival: overwrite that device's slot, then mix frames from your existing core module on all current slots, and broadcast the result.
        - 5. Implement real mic capture (how do i test this)?

---




- **V2:** Move the mix loop onto its own 20ms clock so output rate is fixed at 50 frames per second regardless of how fast or slow chunks arrive.
- **V3:** Implement monitoring and playback after a person leaves the room

    - Implement recording functionality for each individual user

Assumptions:

- Input is 16 kHz, mono, 16-bit PCM, little-endian. Clients convert before sending; the module rejects anything else with a clear error rather than resampling.
- Frames are 20 ms (640 bytes)
- Summing with a hard clamp, not averaging. One speaker alone should sound full volume, so output doesn't scale down as participants join.
- A missing frame means silence for that device, never a stall. Late audio is dropped, not buffered.
- Single process. Rooms aren't shared across instances, so this doesn't scale horizontally as written; Redis or sticky sessions would be the fix.
- No auth, no TLS, no rate limiting
- There is no feedback loop from other devices or speaker when all devices are in the same room
- The developer tests multiple devices by creating multiple tabs to simulate a device
    - How can solo developers test multiple voices by themselves?
        - Maybe we can add preexisting audio files to simulate other people

At first was thinking there would be a central device in the middle of the room to monitor everything. But that wouldn't make any sense since its mentioned that users are using their own devices. So that would mean there cant be an extra device for this task.

**The dict is mixing state living in the wrong place.** Right now the server holds "what each device most recently sent." That's not transport concern, it's mixing concern. As long as it lives in server/, your core module can't actually mix a live conversation on its own — someone integrating it would have to reimplement the slot-holding themselves. That fails the "out of the box" requirement.

It also unblocks the clock. The loop needs to read the slots, so the loop has to live wherever the slots live. Doing A first means B is genuinely just adding a timer, rather than a timer plus a state migration.

**Why push() returning the mix is temporary:** it couples two things that must eventually separate — *recording* a device's frame and *producing* output. As long as one call does both, output timing is a slave to input timing, which is precisely the double-speed defect. When the clock arrives, it will call something like session.mix() at a fixed 50 Hz to produce output, and push() shrinks to a pure store.

**Why drop rather than buffer:** real-time audio has a deadline — a frame is only worth hearing at its moment. An unbounded buffer doesn't save a slow listener's audio; it converts loss into

permanently growing lag: a listener 30 seconds behind a live conversation is hearing noise arranged in the right order, while the server's memory grows without bound. Dropping the *oldest* frame keeps what's still fresh, holds the listener within one second of live, and lets them snap back the moment their connection recovers. The drop counter makes the trade-off observable instead of silent — that's the honest version of "handle the failure path."

**Ceiling 1 — message churn on one event loop.** Everything runs on a single asyncio loop on one core: 50 messages/s in per publisher, 50 out per monitor, so N publishers + M monitors ≈ 50·(N+M) WebSocket message wakeups per second, each with Python-level handler overhead. That's the practical limit — roughly a few hundred total participants before ticks start arriving late (and the resync rule starts eating audio). Bandwidth is minor (256 kbps per stream); it's the per-message CPU that dominates.

**Ceiling 2 — the trap: module-level state means exactly one process.** The obvious "scale it" move, uvicorn --workers 4, doesn't just fail to help — it silently breaks correctness. Each worker gets its own session, its own clock, its own monitors; publishers and monitors land on random workers, so a monitor hears only whichever publishers happened to hash to its process. The single-process constraint is baked in by design (your specs said module-level state each step), but "why can't I just add workers?" is a question you should expect, and the answer is: shared state would have to move out of the process (or rooms get pinned to workers) first.

**Jitter buffers** — the planned next iteration. Slots become small deques. Without this, any frame arriving a few ms late misses its tick and becomes silence, so live mic audio sounds chunky even though the clock is correct.